

The Shape of Meaning: How Hyperbolic Space Discovers What Words Know

There's a problem with the way we represent meaning in machines. We take words, feed them through a model, and get back vectors — points in a high-dimensional space where "king" sits near "queen" and "Paris" sits near "France." This works beautifully for similarity. It fails catastrophically for hierarchy.

A dog is a mammal. A mammal is an animal. An animal is a living thing. This nested structure — the *is-a* relationship that organizes all of human knowledge — doesn't fit in flat Euclidean space. The geometry rebels against it.

But there's a geometry that doesn't.

The Problem With Flat Space

Imagine you're trying to draw a tree on a sheet of paper. The root goes at the top. Each branch splits into children. Each child splits again. By the time you're four levels deep, you have ($2^4 = 16$) leaves. Five levels: 32. Ten levels: 1,024 leaves all demanding their own space at the bottom of the page.

Your paper runs out. The leaves crush together. The hierarchy collapses into a smear.

This is exactly what happens in Euclidean vector space. The volume of a sphere grows as (r^n) — polynomially. But a tree grows *exponentially* with depth. You cannot embed a deep hierarchy in flat space without distorting distances so badly that the structure becomes meaningless. Children end up closer to their cousins than to their parents. The geometry lies.

WordNet: The Tree We're Trying to Draw

Before we solve the geometry problem, we need to understand what we're trying to represent.

WordNet is a lexical database built at Princeton over decades. It's not a neural network or a language model — it's a hand-curated map of how words relate to each other. Its core unit is the **synset**: a set of words that share a meaning. {dog, domestic_dog, Canis_familiaris} is one synset.

{canine, canid} is another, broader one.

Synsets are connected by typed relationships:

- **Hypernym:** X is a kind of Y. Dog → Canine → Mammal → Animal → Organism → Entity.
- **Hyponym:** Y has kinds including X. The reverse direction.
- **Meronym:** X is a part of Y. Finger → Hand.
- **Antonym:** X is the opposite of Y. Hot ↔ Cold.

The hypernym chain forms a vast taxonomic tree. It's one of the most complete maps of conceptual hierarchy ever built – over 117,000 synsets for English, organized into a directed acyclic graph with roughly 200,000 hypernym links.

WordNet tells us what the hierarchy *is*. The question is: how do we represent it geometrically?

Hyperbolic Space: Geometry That Breathes

In Euclidean space, the circumference of a circle grows as $(2\pi r)$. In hyperbolic space, it grows as $(2\pi \sinh(r))$ – which, for large (r) , is approximately (πe^r) . The space *expands exponentially* as you move away from any point.

This is exactly what a tree needs. Near the root, space is tight – there are only a few broad categories. Near the leaves, space is vast – there are millions of specific concepts, and they all need room to be distinct from each other.

The **Poincaré ball** is the standard way to visualize hyperbolic space. It's a unit ball in $(\mathbb{R}^d)$ where the distance between two points isn't the straight line you see – it's given by:

$$d(u, v) = \text{arcosh} \left(1 + 2 \frac{|u - v|^2}{(1 - |u|^2)(1 - |v|^2)} \right)$$

Look at the denominator: $((1 - |u|^2)(1 - |v|^2))$. As a point approaches the boundary $(|u| \rightarrow 1)$, this term goes to zero, and the distance blows up to infinity. The edge of the disk isn't a boundary – it's *infinitely far away*.

This means:

- **Near the origin:** space is small. Points are close together. This is where the root concepts live – "entity," "process," "thing."
- **Near the boundary:** space is enormous. Points can spread out. This is where the leaves live – "golden retriever," "V60 pour-over," "that specific memory from Tuesday."

The geometry naturally encodes abstraction level as radial distance. The more abstract the concept, the closer to the origin. The more specific, the further toward the boundary.

Poincaré Embeddings: Teaching Geometry to Respect Hierarchy

In 2017, Maximilian Nickel and Douwe Kiela at Facebook AI Research showed that you could *learn* embeddings directly in the Poincaré ball. Given WordNet's hypernym relationships, they optimized:

$$\begin{aligned} \mathcal{L} = & \sum_{(u,v) \in \text{hypernyms}} \log \frac{e^{-d(u,v)}}{\sum_{v' \in \text{negatives}} e^{-d(u,v')}} \\ & \end{aligned}$$

This pulls each concept close to its parent while pushing it away from random other concepts. The geometry does the rest. Because hyperbolic space expands exponentially, the optimization naturally places abstract concepts near the origin and specific concepts near the boundary — without being explicitly told to do so.

The results were striking. A 2D Poincaré embedding of WordNet's mammal subtree produces a near-perfect reconstruction of the taxonomic hierarchy. You can see the tree structure in the positions of the points. Siblings cluster together. Parents sit between their children, closer to the origin. The entire hierarchy is visible in the geometry.

Beyond WordNet: Discovering Ontologies You Didn't Know Existed

WordNet is a ground-truth hierarchy, hand-labeled by linguists. But what if you don't have a pre-built ontology? What if you want to *discover* the hierarchy from raw text?

This is where the geometry becomes a discovery tool rather than just a representation.

Step 1: Embed Everything

Take your text — chat messages, documents, entity mentions — and embed them into Euclidean space using any off-the-shelf model. Sentence-BERT, fastText, even the residual stream of a language model. The embedding model doesn't need to know about hierarchy. It just needs to produce vectors where semantically similar things are near each other.

Step 2: Find the Clusters

Run a density-based clustering algorithm like HDBSCAN on the vectors. This finds the natural neighborhoods — groups of concepts that hang together. A cluster might contain `{V60, Chemex, Aeropress, French press}`. The algorithm doesn't know these are coffee brewers. It just sees a dense region of points.

Step 3: Test for Hierarchy

Here's where it gets interesting. For any two clusters, you can test whether one is *contained within* the other using subspace geometry.

Take all the vectors in cluster A. Take all the vectors in cluster B. Run SVD on each to find their effective subspaces — the linear manifolds they occupy. If A's subspace is mostly contained within B's subspace (tested by projecting A's vectors onto B's basis and measuring reconstruction quality), then B is a candidate parent of A.

This works because broader categories span more dimensions. "Mammal" occupies a higher-dimensional subspace than "dog" because it has to accommodate the variation between dogs, cats, whales, and bats. "Dog" is a lower-dimensional slice within that larger manifold. The subspace nesting is the hierarchy.

Step 4: Project Into Hyperbolic Space

Take the Euclidean vectors and project them into the Poincaré ball:

$$\text{poincare}(x) = \tanh(\sqrt{c} \cdot |x|) \cdot \frac{x}{|x|}$$

The projection preserves angles (so cosine similarity still works) but distorts distances in exactly the right way: higher-dimensional, more specific concepts naturally land further from the origin. The radial position becomes a measure of abstraction.

Step 5: Read the Ontology

At this point, the geometry *is* the ontology. You can read hierarchical relationships directly from the positions:

- **Radial distance from origin** = abstraction level. Near the center: broad categories. Near the boundary: specific instances.
- **Angular proximity** = semantic similarity. Things in the same direction are related.
- **Subspace containment** = hypernym relationship. If A's manifold is inside B's, B is a parent of A.
- **Vector offsets** = relational analogies. $(\text{dog} - \text{canine}) \approx (\text{sparrow} - \text{bird})$.

None of this requires a pre-labeled ontology. The geometry discovers the structure that was latent in the distribution of text.

The Subconscious of a System

There's a deeper idea here. When you embed text and let the geometry organize itself, the vector space becomes a kind of **subconscious** for the system.

It doesn't "know" that dogs are mammals. It doesn't reason about taxonomy. It just accumulates statistical regularities from every piece of text it sees, and those regularities deform the space. Clusters form. Subspaces nest. The geometry *is* the memory — not stored as explicit facts, but as distances, curvatures, and manifold boundaries.

When you query it — find the nearest neighbors, test for subspace containment, measure the radial position — you're not computing. You're *perceiving* the shape the data has taken.

This is intuition, in the most literal sense: pattern recognition operating below the threshold of explicit reasoning. Fast, holistic, proposing rather than concluding. The geometry says "these things feel related." It doesn't explain why. It just gives you a number.

What This Enables

Automatic Taxonomy Building

Feed a system your company's internal documents. The geometry discovers that `{quarterly_report, earnings_statement, balance_sheet}` form a cluster, nested within a broader `{financial_documents}` cluster, nested within `{corporate_artifacts}`. No one labeled these categories. The structure emerged from the way people write about them.

Temporal Organization

If you embed states over time — snapshots of a conversation, a project, or a system's memory — the geometry organizes them chronologically. Recent, high-dimensional states spread out near the boundary. Older, compressed states cluster near the origin. "This morning" and "last Tuesday" occupy different radial bands. The passage of time becomes a trajectory through hyperbolic space.

Cross-Lingual Ontologies

Embed the same concepts in English, Japanese, and Arabic. The geometry doesn't care about language — it sees vectors. "Dog," "犬," and "كلب" cluster together because they occupy the same semantic neighborhood. The ontology transcends the vocabulary.

Anomaly Detection

A new concept appears that doesn't fit any existing cluster. Its reconstruction error is high when projected onto every known subspace. The geometry flags it: *this doesn't belong*. It might be noise. It might be something genuinely new. Either way, the system notices.

The Limits

The geometry is not magic. It has real constraints:

It needs density. A concept mentioned three times won't form a detectable cluster. The geometry works with statistical regularities, not one-off facts. Sparse concepts are invisible to it.

It doesn't name things. A cluster is just a set of vectors. The geometry can tell you that {V60, Chemex, Aeropress} belong together, but it can't tell you they're "coffee brewing equipment." Naming requires a language model — something that can look at the members and generate a label.

It doesn't explain relationships. The geometry says "A is contained within B." It doesn't say *why*. Is it a taxonomic relationship (dog → mammal)? A meronomic one (finger → hand)? A temporal one (Tuesday → this week)? The geometry sees the structure but not the semantics.

It can hallucinate structure. Noise in the embeddings, sparse regions, and the curse of dimensionality can create phantom clusters. Not everything the geometry proposes is real. Validation is essential.

The Practical Recipe

If you want to build this yourself, here's the stack:

Component Tool Role

Embedding Sentence-BERT (`all-MiniLM-L6-v2`) Turn text into vectors

Vector index FAISS or USearch Fast nearest-neighbor search

Clustering HDBSCAN Find dense neighborhoods

Subspace analysis NumPy/SciPy (SVD) Test hierarchical containment

Hyperbolic projection ~10 lines of Python Map to Poincaré ball

Naming/validation Any LLM Label the clusters, verify the hierarchy

The geometric pipeline — clustering, subspace containment, projection — is maybe 200 lines of code. It's pure linear algebra. No training, no inference, no GPU. The only learned components are the embedding model (which you're probably already running) and the optional LLM for naming.

Why This Matters

We've spent a decade building systems that understand similarity. "This is like that." Recommendation engines, semantic search, nearest-neighbor retrieval — all variations on the same theme.

But human knowledge isn't just a cloud of similar things. It's organized into trees. Categories contain subcategories. Abstractions sit above instances. The *is-a* relationship is the skeleton of thought.

Hyperbolic geometry gives us a way to represent that skeleton in machines — not by labeling it by hand, but by letting the structure emerge from the data. The geometry doesn't just store the hierarchy. It *discovers* it.

And the discovery process is beautiful in its simplicity: embed, cluster, project, read. The shape of the space tells you what the concepts know about each other. You just have to look.

The Poincaré ball model was introduced by Henri Poincaré in the 1880s as a way to visualize non-Euclidean geometry. He could not have known that 140 years later, his mathematical curiosity would become the natural geometry for representing the structure of meaning itself. But that's the thing about beautiful math — it has a way of finding its purpose.